

Reviewer Pack — Minimal p-Adic Hensel's Lifting Exponent and Resolution Proo...

Entry b6381d0fbb46 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 02:54:15 UTC

Minimal p-Adic Hensel's Lifting Exponent and Resolution Proof Width

Entry ID: b6381d0fbb46 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 02:54:15 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For a given Boolean satisfiability instance φ with n variables, the minimal p-adic Hensel's lifting exponent ($\text{hexp}(\varphi)$) is linearly related to its resolution proof width $w(\varphi)$, such that $\text{hexp}(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

p-adic analysis provides a framework for studying algebraic structures in number fields, and Hensel's lifting is a fundamental tool in p-adic arithmetic. This conjecture suggests that the complexity of resolution proofs could be related to the algebraic properties of Boolean satisfiability instances through the lens of p-adic numbers.

Taxonomy category: p-adic_analysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c7fffa1fd0810c7a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient r between the minimal p-adic Hensel's lifting exponent ($\text{hexp}(\varphi)$) and resolution proof width ($w(\varphi)$) of 30 randomly generated SAT instances meets $r \geq 0.8$ AND the mean absolute difference between $\text{hexp}(\varphi)$ and $w(\varphi)$ is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic analysis AND Boolean satisfiability AND Hensel's lifting exponent - resolution proof complexity AND p-adic Hensel's lifting exponent - Boolean satisfiability instance AND linear relationship AND p-adic Hensel's lifting exponent

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/2408.00810v3>] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [<http://arxiv.org/abs/hep-th/9410058v3>] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [<http://arxiv.org/abs/hep-th/0312046v1>] p-Adic and Adelic Quantum Mechanics - [<http://arxiv.org/abs/1309.3982>] Desingularization of complex multiple zeta-functions, fundamentals of p -adic multiple L -functions, and evaluation of - [<http://arxiv.org/abs/2201.08874v2>] Note on p -adic Local Functional Equation - [<http://arxiv.org/abs/1103.5740v2>] Generating and Searching Families of FFT Algorithms - [<http://arxiv.org/abs/2206.00903v2>] Satisfiability of Quantified Boolean Announcements

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=253.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(n):
            clause = [random.choice(variables), -random.choice(variables)]
            clauses.append(clause)
        return clauses

    def hexp(instance):
        p = 2
        k = 0
        while True:
            found_solution = False
            for clause in instance:
                if all(abs(x) not in {abs(y) for y in clause} for x in range(-p, p+1)):
                    found_solution = True
                    break
            if found_solution:
```

```

        k += 1
        p *= 2
    else:
        return k

def resolution_width(instance):
    clauses = instance[:]
    width = 0
    while clauses:
        new_clause = None
        for i in range(len(clauses)):
            for j in range(i + 1, len(clauses)):
                if abs(clauses[i][0]) == abs(clauses[j][0]):
                    new_clause = [c for c in clauses[i] if c != -clauses[j][0]] + [c for c in clauses[j] if c != -clauses[i][0]]
                    break
            if new_clause:
                break
        if not new_clause:
            return width
        clauses.append(new_clause)
        width = max(width, len(new_clause))
    return width

n_values = [5, 10, 15, 20, 30, 40]
hexp_values = []
widths = []

for n in n_values:
    instance = generate_sat_instance(n)
    hexp_val = hexp(instance)
    width = resolution_width(instance)
    hexp_values.append(hexp_val)
    widths.append(width)

correlation_coefficient = sum((hexp_val - mean_hexp) * (width - mean_width) for hexp_val, width in zip(hexp_values, widths)) / len(hexp_values)
mean_hexp = sum(hexp_values) / len(hexp_values)
mean_width = sum(widths) / len(widths)
mean_abs_diff = sum(abs(hexp_val - width) for hexp_val, width in zip(hexp_values, widths)) / len(hexp_values)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation_coefficient) >= 0.8 and mean_abs_diff <= 3,
    "counterexample": "" if abs(correlation_coefficient) >= 0.8 and mean_abs_diff <= 3 else "n"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]
    results = [run_trial(seed) for seed in seeds]

    correlation_coefficients = [r["metric_value"] for r in results if "metric_value" in r]
    conjecture_holds = all(r["conjecture_holds"] for r in results)

    mean_corr_coeff = sum(correlation_coefficients) / len(correlation_coefficients)

```

```

std_dev_corr_coeff = math.sqrt(sum((x - mean_corr_coeff) ** 2 for x in correlation_coefficient
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

if conjecture_holds:
    print(f"RESULT: SUPPORTED mean={mean_corr_coeff} std={std_dev_corr_coeff} support_fraction={support_fraction}")
elif first_failing_seed is not None:
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the Pearson correlation coefficient and mean absolute difference could not be calculated to verify the conjecture. | next: Retry the experiment with a longer time limit or increase the number of SAT instances to ensure sufficient data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13571
2	preregistration	ollama_remote	glm4:latest	0	0	9955
3	novelty	ollama_remote	glm4:latest	0	0	8661
4	novelty	ollama_remote	glm4:latest	0	0	9862
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33251
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9864
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10097
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12168
9	judge	ollama_remote	glm4:latest	0	0	74521

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 181949 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/b6381d0fbb46.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b6381d0fbb46.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b6381d0fbb46.tar.gz (if generated)